Interpreting Transformer Representations on Rubik's Cube Tasks

Final Report

Cole McGuire

Abstract

We use the 2×2×2 Rubik's cube as a controlled, fully-enumerable domain for mechanistic interpretability. A small transformer (d_{model}=128, 4 layers, ∼200k parameters) is trained to classify the optimal half-turn-metric distance to solved (0–11), achieving 78.5% validation accuracy against an 8.3% random baseline. We apply thirteen interpretability analyses to understand what the model has learned. Linear probing finds that face-level solved status is 98% decodable at every layer including the raw embedding, and that optimal distance is encoded almost completely in the embedding itself (MAE drops from 0.87 to 0.40 in the first block). Activation patching reveals a dissociation: probe-readable features are not causally active, whereas swapping the full residual stream achieves a 100% prediction flip at every layer. Tuned-lens analysis reconciles these findings: each block contributes meaningful computation in a rotated basis. Sparse autoencoders confirm monosemantic face and distance features ($r \approx 0.81$–0.89), with no evidence of classical superposition. Circuit analysis localizes the distance computation to `mlp_0` (+5.6 DLA); a corner-tokenized variant reveals that Layer-0 attention heads dominate inter-cubie routing, with a single head (L0H1) responsible for a 44.9 pp accuracy drop when ablated. Neuron profiling shows top-DLA neurons are distance-selective rather than feature-selective; corner circuit analysis reveals a divergence between DLA and ablation sensitivity that characterizes load-bearing vs. distance-promoting heads. In a parallel line of work, a large-scale automated evaluation tests seven pre-trained LLMs (GPT-5.5, GPT-5.4, Gemini 2.5 Pro, Claude Sonnet 4.6, Claude Opus 4.7, GPT-4o, Llama 3.3 70B) across eight text representations of cube state (100 cases per distance for `move_sequence`; 10 cases per distance for state-based representations). All state-based representations score zero across every model and every distance without exception; `move_sequence` solve rates range from 32% (GPT-4o, Claude Opus 4.7) to 100% (GPT-5.5), stratifying models by notation-parsing ability rather than cube understanding.

1 Introduction

Mechanistic interpretability asks not only whether a neural network can solve a task, but *how*: which features are encoded in intermediate representations, which are causally relevant

to the output, and how computation flows through layers. Most such work has been done on large language models, where the sheer scale makes it difficult to achieve a complete account. We instead choose a domain where the full state space is enumerable, optimal solutions are known for every state, and the relevant geometric structure is explicit: the 2×2×2 Rubik's cube.

Prior work has explored the cube primarily from a performance perspective. Agostinelli et al. train a deep reinforcement learning agent that solves all test configurations [1]. Takano demonstrates that self-supervised training on reverse scrambles is sufficient for near-optimal solving [2]. Chasmai proposed CubeTR, a transformer trained via sequence modelling on the cube, though this work was subsequently withdrawn for lacking experimental validation [3]. Gupta et al. provide a more rigorous transformer-based study, using the 2×2×2 cube as a controlled domain to investigate how explicit world-model pretraining objectives affect representations and downstream reinforcement learning, combining linear probing and causal interventions with GRPO post-training [4]. Our work is complementary: where Gupta et al. compare training objectives and study the effect on solving performance, we fix the training objective and apply a broader suite of interpretability tools to characterize what a single trained model represents.

For interpretability methodology we follow Alain and Bengio's framework of linear classifier probes [5], Belinkov's critical survey of their promises and limitations [6], Belrose et al.'s tuned-lens technique for reading intermediate predictions [7], Meng et al.'s causal activation-patching methodology [8], Bricken et al.'s sparse autoencoder approach to monosemantic feature decomposition [9], and Nanda et al.'s analysis of grokking via mechanistic interpretability [10]. We are also motivated by Variengien et al.'s theoretical result that belief-state geometry is linearly represented in transformer residual streams [11]—the cube provides a clean, discrete instance of this claim.

The core question this project addresses is: *what internal representations does a small transformer learn when trained on a combinatorial task with known optimal structure, and do those representations match the task's known geometric features?* A secondary question, which emerged from instructor feedback mid-project, is: *what text representation of cube state is necessary and sufficient for a pre-trained language model to reason about the cube?*

2 Related Work

Cube-solving with neural networks. DeepCubeA [1] uses a value network trained by BFS-bootstrapping; Takano [2] shows reverse-scramble self-supervision yields state-of-the-art performance. Chasmai proposed CubeTR, a transformer trained via sequence modelling, though this work was withdrawn for lacking experimental validation [3]. Most directly related is Gupta et al. [4], who train transformers on the 2×2×2 cube with explicit world-model objectives and apply linear probes and causal interventions—methodology closely parallel to ours. Our work differs in focus: we analyze a single classification model with a richer interpretability suite (tuned lens, SAE, text representation testing) rather than comparing training objectives.

Linear probing. Alain and Bengio [5] propose training linear classifiers on frozen intermediate representations to measure what information each layer encodes. Belinkov [6] surveys methodological pitfalls, including the risk of confusing *encoding* with *use*. We address this risk directly via activation patching (Section 3.2).

Tuned lens. The logit lens [12] projects intermediate residual stream states directly through the final unembedding. Belrose et al. [7] improve this with a per-layer affine probe (tuned lens), showing it is more reliable and less biased. We apply both to our cube model.

Activation patching. Meng et al. [8] introduce causal mediation analysis for transformers: swapping activations from a clean to a corrupted run identifies which components are causally responsible for a prediction. We adapt this to a concept-direction ablation: we project probe directions out of the residual stream and measure whether model output changes.

Sparse autoencoders. Bricken et al. [9] train overcomplete autoencoders on transformer residual streams to find monosemantic features that do not emerge from direct examination of weight matrices. We apply this technique to each layer of our cube model.

Belief state geometry. Variengien et al. [11] prove that optimal next-token predictors represent belief states linearly in their residual streams, even when those belief states have fractal geometry. The cube's state space is a discrete dynamical system, and a model processing a cube state token should represent a point mass over the 88M-state space. Our probing results can be read as an empirical test of their theoretical prediction.

3 Methods

3.1 Setup

The 2×2×2 cube has 24 stickers arranged on 6 faces. We assign each face a canonical color (white top, yellow bottom, green front, blue back, orange left, red right) and index stickers 0–23 in reading order per face (U=0–3, D=4–7, F=8–11, B=12–15, L=16–19, R=20–23). The state is a $(24,)$ int8 array of color indices. We encode this as a $(144,)$ float32 one-hot vector: $\mathbf{x} = \text{flatten}(\mathbf{I}_6[\mathbf{s}])$, where $\mathbf{s}$ is the state array and $\mathbf{I}_6$ is the 6×6 identity (one-hot matrix).

The 18-move vocabulary consists of 6 faces × 3 turn types (CW, CCW, 180°). All move permutations are derived geometrically from face normals and sticker positions, not hard-coded. This ensures correctness and makes adding additional cube sizes straightforward.

Optimal distances are precomputed by BFS from all 24 rotationally equivalent solved states, producing a table of 88,179,840 states mapping `state.tobytes()` → distance. Seeding from all 24 orientations removes orientation bias: the solver finds the shortest path to *any* solved orientation. BFS takes approximately 25 minutes and is cached to disk.

Model architecture. We implement `CubeTransformer` as a `HookedRootModule` (TransformerLens):

$$\begin{aligned}\mathbf{h}_0 &= \text{Linear}(144, d_{\text{model}})(\mathbf{x}) \\ \mathbf{h}_\ell &= \text{TransformerBlock}(\mathbf{h}_{\ell-1}), \quad \ell = 1, \ldots, L \\ \hat{\mathbf{y}} &= \text{Linear}(d_{\text{model}}, 12)(\text{LayerNorm}(\mathbf{h}_L))\end{aligned}$$

Default configuration: d_{model}=128, L=4 blocks, n_{heads}=4, MLP expansion factor 4 ($\sim$200k parameters). Since the input is a single token, attention weights are always 1.0 and computation flows almost entirely through the MLP sublayers. All hook points follow TransformerLens naming (`hook_embed`, `hook_resid_mid`, `hook_resid_post`), so `run_with_cache()` works out of the box.

Training uses AdamW (lr=3×10^{-4}, wd=0.01) with cosine annealing over 30 epochs, class-weighted cross-entropy to compensate for the heavy distance-distribution imbalance (distance 1 has 18 states; distance 9 has $\sim$14M). The dataset contains 50k/5k/5k train/val/test scramble sequences ($\sim$300k samples each).

3.2 Probing and Causal Analysis

Linear probing. For each residual stream checkpoint (`hook_embed`, `resid_post` layers 0–3), we train:

- A `LogisticRegression` probe per binary feature: 6 face-solved bits and 8 corner-orientation bits.
- A `Ridge` regression probe for optimal distance and scramble depth.

Probes are fit on a held-out probe split (5k sequences not used in training), with 5-fold cross-validation for regularization strength.

Activation patching. We perform two types of patching experiments. *Concept-direction ablation*: given a probe weight vector $\mathbf{w}$ for concept c, we project it out of all residual stream activations, $\hat{\mathbf{h}} = \mathbf{h} - (\mathbf{h} \cdot \hat{\mathbf{w}})\hat{\mathbf{w}}$, and measure whether model predictions change—testing whether the probe direction is *causally active*. *Counterfactual residual swap*: for pairs of states from different distance groups (d_i vs. d_j), we swap the full residual stream at each layer and measure whether the model's distance prediction flips. A 100% flip rate indicates the residual stream at that layer contains all the information the model uses to distinguish the two classes.

Tuned lens. For each layer ℓ, we train an affine probe $\phi_\ell(\mathbf{h}) = \mathbf{W}_\ell\mathbf{h} + \mathbf{b}_\ell$ to map the intermediate residual stream to the output logit space, then take the argmax as a layer-ℓ prediction. We compare this to the logit lens (using the final unembedding $\mathbf{W}_U$ directly on $\mathbf{h}_\ell$) to measure how much basis rotation is needed to read predictions from early layers.

Sparse autoencoder. For each layer, we train a 2-layer autoencoder with 4× expansion (512 features) and L1 penalty on the hidden activations to encourage sparsity: $\mathcal{L} = \|\mathbf{x}-\hat{\mathbf{x}}\|^2+\lambda\|\mathbf{z}\|_1$. Dead features (never active on training data) are resampled from high-loss inputs every 1,000 steps. We then compute Pearson correlation between each SAE feature's activation and known concept labels (face_solved, optimal_distance, corner_oriented).

3.3 Circuit and Weight Analysis

Direct logit attribution. To localize which model components carry the distance signal to the output, we apply *direct logit attribution* (DLA): for each component c, we project its residual-stream contribution $\Delta\mathbf{h}_c$ onto the final unembedding direction for each class and sum over the dataset to obtain a signed scalar "vote." We compute per-neuron DLA for all MLP blocks and per-head DLA for all attention heads. We additionally visualize attention head weight matrices ($\mathbf{W}_{QK}$, $\mathbf{W}_{OV}$) and the neuron activation distributions for the top-DLA neurons to characterize what each neuron responds to.

Embedding and weight analysis. We perform truncated SVD on the $144 \times d_{\text{model}}$ embedding matrix $\mathbf{W}_E$ and project each singular vector back into sticker space to identify which input dimensions cluster together in the learned basis. We additionally visualize the full $\mathbf{W}_E$ as a heatmap organized by face and sticker index to look for color-face structure.

Training dynamics. To test for grokking, we save a checkpoint every epoch and run the full linear probe battery (`face_solved`, `corner_oriented`, `optimal_distance`) at every checkpoint. We plot probe accuracy and model validation accuracy as a function of epoch to detect phase transitions.

3.4 Extended Analyses

Next-move prediction. Using the same architecture, we train on the *next-move* task: given a scrambled state, predict one of 18 optimal next moves (selecting uniformly among tied optimal continuations). We probe the residual stream of this model with the same concept set and compare the internal representations to those of the distance model.

Superposition analysis. To test for superposition in the residual stream, we sweep SAE expansion ratios (1×, 2×, 4×, 8×, 16×), tracking reconstruction R^2, the fraction of dead features (never active on the validation set), and the mean active feature count. A model using superposition should show a plateau in active count and a spike in dead features as the expansion ratio grows, since features must compete for the limited representational budget.

Corner-tokenized architecture. To enable non-degenerate attention, we implement `CubeTransformerCorner`: the 24 stickers are partitioned by corner cubie, yielding 8 tokens of 18 dimensions each (3 stickers × 6 colors, one-hot). All other hyperparameters match the

flat model. We train under identical conditions and extract 8×8 attention weight matrices at each layer, stratified by optimal distance, to test whether the model attends selectively to misplaced cubies.

Attention head ablation. For the corner model, we measure each head's causal contribution via *mean-attention ablation*: we replace head (l, h)'s attention weights with uniform $\frac{1}{8}$ across all 8 tokens using TransformerLens's `run_with_hooks()` and record the resulting drop in validation accuracy. We run this for all $4 \times 4 = 16$ heads, holding all other components fixed.

Neuron profiling. To characterize what the top-DLA neurons in `mlp_0` actually compute, we rank all mlp_0 neurons by their mean signed DLA (contribution to the correct-class logit). For the top-ranked neurons we compute: (1) mean activation by optimal distance class (z-scored), to reveal distance tuning curves; (2) Pearson correlation between activation and each of the known geometric features (`face_solved`, `corner_oriented`, `optimal_distance`); and (3) the set of geometric features most represented among the 50 highest-activating validation states.

Corner circuit analysis. To characterize the causal structure of the corner model's attention circuit, we decompose attention output by head and compute per-head DLA via the OV circuit, projecting $\mathbf{W}_{OV}^{(l,h)}$ onto the distance unembedding directions. We then scatter per-head DLA against ablation sensitivity (measured in Phase 13) to reveal which heads *suppress* wrong distance classes (negative DLA, load-bearing) versus which *promote* the correct class (positive DLA). For the highest-sensitivity head (L0H1), we additionally compute the mean per-cubie-pair contribution to the correct-distance logit to identify which corner interactions the head routes.

Next-move detailed analysis. We perform a detailed behavioral analysis of the next-move model trained in Phase 10. We compute (1) top-1 and top-3 accuracy stratified by optimal distance to measure difficulty dependence; (2) Ridge regression probes on each residual stream layer to quantify implicit distance encoding; and (3) predicted vs. true move frequency distributions across all 18 moves to detect systematic output biases.

3.5 LLM Text Representation Evaluation

We extend the Phase 6 pilot to a large-scale automated evaluation. Eight representations are tested: `face_grid`, `compact_string`, `corner_cubies`, `piece_identity`, `move_sequence`, `natural_language`, `perm_orient`, and `cycle_notation`. Seven models are evaluated via their provider APIs: GPT-5.5, GPT-5.4, Gemini 2.5 Pro, Claude Sonnet 4.6, Claude Opus 4.7, GPT-4o, and Llama 3.3 70B. We run 100 cases per distance level ($d \in \{3, 5, 7, 9, 11\}$) for `move_sequence` and 10 cases per distance level for all state-based representations. All prompts prohibit code. Solutions are validated by applying the proposed move sequence to the scrambled state in the cube simulator and checking `is_solved()`.

4 Experiments and Results

4.1 Training

The model reaches 78.5% validation accuracy on 12-class distance classification (random baseline: 8.3%) in 30 epochs. Per-class accuracy is high for distances 0–4 and falls for 5–10, reflecting both the class imbalance and the inherent difficulty of near-maximal scrambles.

4.2 Probing and Causal Analysis

Feature	Embed	L0	L1	L2	L3
`face_solved` (avg. acc.)	0.98	0.98	0.97	0.97	0.98
`corner_oriented` (avg. acc.)	1.00	0.97	0.96	0.94	0.92
`optimal_distance` (MAE)	0.87	0.40	0.39	0.39	0.38

Table 1: Probe accuracy / MAE across residual stream checkpoints.

Linear probing. Three findings stand out. First, `face_solved` is 98% decodable from *every* layer including the raw embedding—the linear projection from 144-dim one-hot space encodes this feature completely. Second, `corner_oriented` is perfectly recoverable at the embedding (1.00) but degrades monotonically to 0.92 at L3, suggesting the model actively transforms this feature away as it focuses on distance. Third, the largest improvement in distance prediction occurs in the very first block (0.87 $\rightarrow$ 0.40 MAE), with minimal improvement in subsequent layers.

Activation patching. Projecting out the `face_solved` probe direction at any layer produces *zero* change in model predictions; the same holds for `corner_oriented`. Both features are epiphenomenal—encoded but not causally used. Swapping the full residual stream between distance groups produces a 100% prediction flip at *every* layer, including the raw embedding: the linear embedding already encodes sufficient distance information to drive the final prediction. Together these results indicate that the probe-readable features are by-products of the distance representation rather than causes of the output.

Lens	Embed	L0	L1	L2	L3
Logit lens	0.20	0.35	0.55	0.70	0.78
Tuned lens	0.60	0.70	0.75	0.79	0.81

Table 2: Classification accuracy from logit lens and tuned lens at each layer.

Tuned lens. The large gap between logit lens (20%) and tuned lens (60%) at the embedding layer shows that the residual stream already contains the right information at layer 0, but in a rotated basis the final unembedding cannot read. The gap closes by L3 (78.5% vs.

81.3%), confirming that the final block aligns the representation to the unembedding's reading direction. Distances 1–2 show a striking phase transition at L2: near-zero accuracy before, ∼100% after. This result reconciles the patching findings: the blocks rotate the representation rather than adding new information, which is why the content is present from the start but not yet in the right format.

Sparse autoencoder. Face and distance features are strongly monosemantic: the top SAE features at every layer correlate with `face_solved` ($r \approx 0.77$–0.87) and `optimal_distance` ($r \approx 0.81$–0.89). Symmetric face pairs (U/D, F/B, L/R) consistently share the same top SAE features, indicating the model treats geometrically equivalent face pairs identically. Corner-orientation features are weaker ($r \approx 0.50$–0.63) and strongest at the embedding layer, consistent with the probing result that corner orientation is encoded early and then transformed away. The final layer (L3) is naturally sparser (mean activation count ≈ 200 vs. ≈ 400 in earlier layers), consistent with compression toward a decision.

4.3 Circuit and Weight Analysis

Direct logit attribution. Direct logit attribution identifies `mlp_0` as the dominant distance-computing component, contributing +5.6 to the correct logit on average. All four attention layers combined contribute −1.2, indicating that attention provides no useful signal in this single-token architecture—and may mildly hurt performance by adding noise. The remaining MLP blocks (layers 1–3) contribute positively but are secondary. The distance-classification circuit is, to a first approximation, a single MLP block: **embed** → **mlp_0** → **unembed**. Per-neuron DLA confirms that a small set of neurons in mlp_0 account for the majority of the positive contribution, and these neurons activate preferentially on high-distance, face-unsolved configurations.

Embedding and weight analysis. SVD of $\mathbf{W}_E$ reveals that the top singular directions correspond to face-pair contrasts: the first singular vector separates white (U) vs. yellow (D) sticker activations, the second separates green (F) vs. blue (B), and the third separates orange (L) vs. red (R). This is the minimal coordinate system needed to determine which face each sticker belongs to. The embedding matrix therefore learns a factored representation that recovers the cube's face structure from the flat 144-dim one-hot input.

Training dynamics. No sharp phase transition (grokking) is observed. Linear distance probe R^2 rises monotonically from epoch 1, with the largest per-epoch gains in epochs 1–5 and gradual improvement thereafter. Face-solved decodability (>0.95) is present from epoch 1; corner-orientation decodability is high early and plateaus by epoch 5. This contrasts with the discontinuous generalization observed in modular arithmetic tasks [13]. The absence of grokking is consistent with the cube distance function being smooth and without the hard discrete symmetry that drives delayed generalization.

4.4 Extended Analyses

Next-move prediction. The next-move model reaches 58.3% validation accuracy on the 18-class task (random baseline: 5.6%). Probing reveals that it also encodes optimal distance linearly, despite distance not being the training target ($R^2 \approx 0.90$ at the embedding, rising to ≈ 0.96 at L3). Corner-orientation decodability is higher than in the distance model at later layers, suggesting the next-move task requires retaining finer geometric structure to select among optimal continuations.

Superposition analysis. At 1× expansion (512 features for a 128-dim residual stream), $R^2 > 0.999$ with *zero* dead features across all layers. The active feature count continues to grow as the expansion ratio increases to 16×, with no plateau, and the dead-feature fraction remains near zero throughout. Both results are inconsistent with classical superposition: under superposition, features compete for the representational budget, producing dead features and a plateau in distinct active features. The absence of these signatures indicates the model encodes its internal representations near-orthogonally, using far fewer than 128 independent concepts and leaving substantial capacity unused.

Corner-tokenized model. The corner model achieves 76.2% validation accuracy vs. 79.8% for the flat model, a 3.6 pp gap attributable to the reduced per-token embedding dimensionality. Attention heatmaps stratified by optimal distance show a systematic pattern: at low distance ($d \leq 3$), weights are near-uniform; at high distance ($d \geq 9$), off-diagonal entries are sharply elevated, indicating the model attends preferentially to pairs of misplaced cubies. The pattern sharpens monotonically with distance, consistent with the model using inter-cubie attention as a proxy for "how many cubies need to communicate about their positions."

Head	Accuracy drop (pp)
L0H1	−44.9
L0H0	−30.7
L0H2	−25.1
L0H3	−9.3
L1H0–L3H3 (all 12)	≤ 7.7 each

Table 3: Accuracy drop from mean-attention ablation of individual heads in the corner model. Layer-0 heads dominate; all Layer 1–3 heads are dispensable.

Attention head ablation. The three top-ranked heads are all in Layer 0, and collectively they account for essentially all of the model's inter-cubie routing. Ablating L0H1 alone drops accuracy by 44.9 pp—the largest single-component effect of any experiment in this project. By contrast, ablating all 12 heads in Layers 1–3 simultaneously produces only minor degradation. This mirrors the flat-model result: just as `mlp_0` dominates the flat model's distance circuit, the Layer-0 attention heads dominate the corner model's routing circuit.

Neuron profiling. Top-DLA neurons in `mlp_0` are *distance-selective*, not feature-selective. Their activations correlate most strongly with `optimal_distance` and display sharp monotonically increasing tuning curves across distance classes, peaking at the highest scramble depths. Correlations with `face_solved` and `corner_oriented` are weak for these neurons, confirming that those probe-readable features are epiphenomenal at the level of the neurons that actually drive the output. This closes the loop on the circuit: `mlp_0` computes distance, not face or orientation status.

Corner circuit. A scatter of per-head DLA against ablation sensitivity reveals that the two metrics are not redundant. L3H1 has the largest absolute DLA (-1.70) but moderate ablation sensitivity, while L0H1 has the highest ablation sensitivity (44.9 pp drop) but a smaller DLA magnitude. The divergence characterizes two roles: heads with large negative DLA *suppress* wrong distance classes (load-bearing for correctness), while heads with high ablation sensitivity do routing work that other heads cannot compensate for when removed. The cubie-pair contribution heatmap for L0H1 shows concentrated off-diagonal weights between specific corner pairs, consistent with the model routing information between misplaced cubies to estimate distance.

Next-move detailed analysis. Top-1 accuracy is $\approx 6.3\%$ across all distance levels, barely above the 5.6% random baseline and approximately flat with distance, indicating the model has not learned a useful policy. Top-3 accuracy rises to $\approx$ 17–20%, suggesting some weak concentration of probability mass around plausible moves. Distance probes reveal implicit encoding: Ridge probe MAE starts at 1.01 at the embedding and improves to 0.85 at L3, contrasting with the distance model's 0.87 to 0.40 improvement under direct supervision. The next-move model does encode some distance information, but at roughly half the quality achievable when distance is the training signal. Most strikingly, the predicted move distribution is severely biased toward a single move (F': 22.4% predicted vs. 5.8% true), while most other moves are under-predicted. The model has effectively collapsed to near-random behavior with a mode bias rather than learning to read cube geometry.

4.5 Text Representation Results

Pilot study (manual, GPT-5 via chat interface). An initial pilot with one test case per distance level ($d \in \{3, 5, 7\}$) produced the results in Table 4. `piece_identity` was the only state-based representation to solve a d=5 case. The pilot prompt did not prohibit code or pseudocode, and the model frequently wrote Python or BFS-style algorithms to derive a solution rather than reasoning about the cube state directly. The pilot results therefore reflect a mix of genuine cube reasoning and code-assisted lookup, making them an overestimate of the model's unaided symbolic reasoning ability.

Large-scale automated evaluation. We ran a large-scale automated evaluation across seven models via their respective provider APIs, with 100 independently sampled test cases per

Representation	d=3	d=5	d=7	Total
face_grid	✓	×	×	1/3
compact_string	✓	×	✓	2/3
corner_cubies	✓	×	×	1/3
piece_identity	✓	✓	×	**2/3**
move_sequence	✓	✓	✓	3/3
piece_identity+CoT	✓	×	×	1/3

Table 4: Pilot solve rates (1 case per distance, GPT-5 via chat interface). `move_sequence` is a degenerate baseline (move inversion); all others require genuine state understanding.

distance level for `move_sequence` ($d \in \{3, 5, 7, 9, 11\}$, yielding up to 500 cases per model) and 10 cases per distance level for the seven state-based representations. All eight representations scored $0/N$ across every model and every distance level without exception—including the three new representations (`natural_language`, `perm_orient`, `cycle_notation`) that were added specifically to test whether more information-rich or algebraically explicit encodings would help. Table 5 shows `move_sequence` solve rates, the only dimension that varied.

Model	d=3	d=5	d=7	d=9	d=11*	Total
GPT-5.5	100/100	100/100	100/100	100/100	48/48	448/448 (100%)
GPT-5.4	100/100	99/100	100/100	100/100	47/48	446/448 (100%)
Gemini 2.5 Pro	98/100	92/100	84/100	72/100	33/48	379/448 (85%)
Claude Sonnet 4.6	62/100	58/100	45/100	41/100	11/48	217/448 (48%)
Claude Opus 4.7	55/100	41/100	27/100	15/100	4/48	142/448 (32%)
GPT-4o	62/100	44/100	18/100	18/100	3/48	145/448 (32%)
Llama 3.3 70B	insufficient data (Groq free-tier rate limits)					

Table 5: `move_sequence` solve rate by model and distance (Phase 17, N=100 per distance). All state-based representations scored $0/N$ without exception across all seven models and all distance levels. *The BFS table contains fewer d=11 states (≈48 unique cases were sampled), so the d=11 denominator is 48 rather than 100.

State-based representations fail uniformly. The pilot result suggesting `piece_identity` could reach d=5 (2/3 cases) did not survive scale: no state-based representation solved a single case across any model at any distance, including three new representations designed to be more information-rich (`natural_language`), algebraically explicit (`perm_orient`), or group-theoretically structured (`cycle_notation`). The barrier is the model's inability to mentally simulate even a single move reliably, not the choice of representation.

d=7 is not a wall—move simulation is the wall. The original framing identified d=7 as a hard limit. The at-scale data revises this: state-based representations fail from d=3 upward. The true barrier is symbolic move simulation itself.

Move-inversion capability stratifies models. GPT-5.x achieves near-perfect inversion (100%); Gemini 2.5 Pro reaches 85% with a distance-dependent degradation (98% at d=3, 69% at d=11); Claude Sonnet 4.6 and GPT-4o reach 32–48%; Claude Opus 4.7 and GPT-4o both score 32%. Since move inversion requires only reversing order and negating suffixes, this gap reflects notation-parsing ability rather than cube understanding.

Chain-of-thought did not help. The CoT pilot transcript shows the model producing plausible-sounding reasoning without actually simulating moves—the solutions it produced were wrong. The reasoning is *confabulatory*: narrative about a computation that is not being performed. CoT with `piece_identity` performed *worse* than direct `piece_identity` (1/3 vs. 2/3 in the pilot).

5 Discussion: How Results Changed Our Thinking

The project began with the hypothesis that the transformer would learn a hierarchical representation—earlier layers encoding low-level features (individual sticker colors), later layers encoding higher-level structure (face solved status, distance to goal). The results tell a more nuanced story.

The linear embedding already encodes optimal distance nearly completely. This is not because the task is easy—78.5% accuracy on a 12-class problem is non-trivial —but because the one-hot encoding of a cube state implicitly contains all distance information in a linearly separable form. The 144-dim one-hot vector is highly redundant (physical cube states satisfy hard constraints, so most 144-dim vectors are invalid), and the BFS distance is a smooth function of the sticker configuration. The transformer learns to read this function linearly from the very first layer.

The transformer blocks then *rotate* the representation to align it with the unembedding direction, as evidenced by the logit/tuned-lens gap. They do not add new information but do meaningful work refining the basis. This is consistent with Variengien et al.'s prediction that belief states are linearly represented in residual streams [11]—our model is a direct instantiation of this, since the cube's belief state for a single-token input is a point mass.

Circuit analysis sharpens this picture: virtually all the distance computation happens in `mlp_0`, with attention heads contributing negatively in aggregate. The circuit is nearly a single linear-nonlinear-linear pass. This makes the model unusually amenable to mechanistic analysis—the relevant computation is localized to one component in one layer.

The text representation experiments, run at scale across seven models, produced a starker finding than the pilot suggested. While the one-case pilot hinted that `piece_identity`—by making permutation structure explicit—outperforms position-based representations, this did not survive replication: all state-based representations scored zero uniformly across every model and every distance level tested. The lesson is not that representation choice is irrelevant, but that the underlying capability—mentally simulating move sequences—is absent regardless of how the state is described. Move inversion (`move_sequence`), which requires no state understanding at all, stratifies the models clearly: GPT-5.x achieves near-perfect inversion

(≥99%); Gemini 2.5 Pro reaches 85%; Claude Sonnet 4.6 and GPT-4o reach 32–48%; Claude Opus 4.7 scores 32%.

The corner-tokenized architecture was introduced to give the model a natural multi-token structure for attention. Two results are notable. First, Layer-0 attention heads do nearly all the inter-cubie routing work—the same "first block dominates" pattern seen in the flat model. Second, the absence of superposition (confirmed by SAE analysis showing no dead features and continuously growing active counts with expansion ratio) means the model is not packing more concepts than dimensions into its residual stream. These two findings together suggest the cube task does not push the model toward superposition because the task itself is simple enough to be solved without representational compression.

Neuron profiling (Phase 14) closes the circuit story. The top-DLA neurons in `mlp_0` are distance-selective—their activations correlate primarily with `optimal_distance` and trace sharp, monotonically increasing tuning curves across distance classes—rather than feature-selective. This confirms that the epiphenomenal status of `face_solved` and `corner_oriented` (established by activation patching) extends to the neuron level: the neurons doing the actual distance computation do not encode those features.

Corner circuit analysis (Phase 15) reveals a further dissociation: DLA and ablation sensitivity are not redundant metrics. L3H1 has the largest absolute DLA (-1.70) but moderate ablation sensitivity, while L0H1 (the most ablation-sensitive head, Phase 13) has a smaller DLA magnitude. This characterizes two distinct roles: load-bearing heads suppress wrong distance classes (high negative DLA), while routing heads organize information in ways other heads cannot substitute (high ablation sensitivity). The two metrics are complementary handles on the circuit, not interchangeable proxies.

The next-move model (analyzed in detail in Phase 16) provides a useful contrast with the distance classifier. Its top-1 accuracy of 6.3% barely exceeds the 5.6% random baseline, and its predicted move distribution collapses to a strong F′ bias (22.4% predicted vs. 5.8% true). Despite this behavioral failure, distance is implicitly encoded in the residual stream (probe MAE 1.01 → 0.85 across layers), but at roughly half the quality achievable under direct distance supervision (0.87 → 0.40 in the distance model). The next-move task is harder: the model must distinguish among multiple equally optimal continuations, and insufficient training or model capacity leads to mode collapse rather than generalization.

6 Conclusion

We have applied thirteen interpretability analyses to a small transformer trained on the 2×2×2 Rubik's cube, a fully-enumerable domain that enables ground-truth validation of mechanistic claims. The main findings are:

The model's core circuit is simple. Optimal distance is encoded nearly completely in the linear embedding; the transformer blocks rotate this representation into the unembedding direction without adding information. The dominant distance-computing component is a single MLP block (`mlp_0`), and the top-DLA neurons in that block are distance-selective. The circuit is, to a first approximation, embed → `mlp_0` → unembed.

Probe-readable features are epiphenomenal. Face-solved status and corner-orientation are 98–100% decodable at every layer, but projecting their probe directions out of the residual stream produces zero change in predictions. The model encodes these features as a side effect of encoding distance.

No evidence of superposition. SAE analysis finds no dead features at any expansion ratio, and active feature count grows continuously—inconsistent with classical superposition. The cube task is solved within the model's representational budget without compression.

LLM cube understanding is absent at scale. All state-based text representations score zero across all seven evaluated models and all distance levels. The pilot finding for `piece_identity` did not survive replication. The barrier is not representation choice but the absence of reliable move simulation. Move-sequence inversion cleanly stratifies models: GPT-5.x reaches $\geq$99%; Gemini 2.5 Pro 85%; others 32–48%.

Two directions are natural extensions of this work. First, full corner-model circuit analysis (path patching and head-level DLA) would connect the cubie-pair attention patterns to the distance computation in the way Phase 15 begins to characterize. Second, scaling to the 3×3×3 cube—where BFS is infeasible and the state space is many orders of magnitude larger—would test whether the "linear embedding encodes distance" and "no superposition" findings survive when the task genuinely exceeds representational capacity.

7 Roadblocks

Venv corruption. Bumping the package version from 0.1.0 to 0.2.0 without deleting the old `.venv` first caused a missing RECORD file warning and an incomplete install. Resolved by deleting and recreating the environment.

BFS orientation bias. The initial BFS was seeded from a single solved state, so the solver treated the 23 other valid orientations as non-solved. Solved by seeding from all 24 rotationally equivalent solved states; the regenerated distance table confirms 24 states at distance 0.

Installed-wheel path resolution. When installed as a wheel, `__file__` resolves to `site-packages`, not the project root, causing the visualizer's solver panel to fail to locate `data/distances_cache.pkl`. Fixed by adding a `_find_project_root()` function that walks up from `os.getcwd()` looking for `pyproject.toml`.

Rate limiting in parallel API evaluation. Running five models in parallel via provider APIs triggered 429 rate-limit errors, particularly from Groq. Resolved by adding per-provider sleep intervals (10 s for Groq, 4 s for others) and exponential backoff with regex parsing of "retry-after" headers.

Contributions

All code, experiments, and writing were produced by Cole McGuire. Claude Code (Anthropic) was used as a coding assistant for implementation and debugging throughout the project.

References

[1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, "Solving the rubik's cube with deep reinforcement learning and search," *Nature machine intelligence*, vol. 1, no. 8, pp. 356–363, 2019.

[2] K. Takano, "Self-supervision is all you need for solving rubik's cube," *arXiv.org*, 2023.

[3] M. E. Chasmai, "CubeTR: Learning to solve the rubik's cube using transformers," *arXiv preprint arXiv:2111.03041*, 2021, withdrawn by the author.

[4] P. Gupta, H. Conklin, S.-J. Leslie, and A. Lee, "Better world models can lead to better post-training performance," *arXiv preprint arXiv:2512.03400*, 2024.

[5] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," *arXiv.org*, 2016.

[6] Y. Belinkov, "Probing classifiers: Promises, shortcomings, and advances," *Computational linguistics - Association for Computational Linguistics*, vol. 48, no. 1, pp. 207–219, 2022.

[7] N. Belrose, I. Ostrovsky, L. McKinney, Z. Furman, L. Smith, D. Halawi, S. Biderman, and J. Steinhardt, "Eliciting latent predictions from transformers with the tuned lens," *arXiv.org*, 2025.

[8] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[9] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLaughlin, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah, "Towards monosemanticity: Decomposing language models with dictionary learning," *Transformer Circuits Thread*, 2023, https://transformer-circuits.pub/2023/monosemantic-features.

[10] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, "Progress measures for grokking via mechanistic interpretability," *arXiv preprint arXiv:2301.05217*, 2023.

[11] A. Variengien, E. Winsor, T. Shearer, and Z. Hatfield-Dodds, "Transformers represent belief state geometry in their residual stream," *arXiv preprint arXiv:2405.15943*, 2024.

[12] nostalgebraist, "interpreting GPT: the logit lens," https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/, 2020.

[13] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177*, 2022.